Kubernetes Student Information System – Learning Notes & Command Reference

1. Project Overview

The project consists of:

- Spring Boot REST API (Student Information Service)
- MySQL Database
- Docker Containerization
- Kubernetes Deployment on GKE (Google Kubernetes Engine)
- Horizontal Pod Autoscaler (HPA)
- Persistent Storage using PVC
- Ingress for External Access

Architecture:

Internet ↓ Ingress ↓ student-api-service (ClusterIP) ↓ Spring Boot Pods (4 replicas) ↓ mysql-service (ClusterIP) ↓ MySQL Pod ↓ Persistent Volume Claim ↓ Google Persistent Disk

---

2. Application Build & Docker Commands

Build Spring Boot Application

```
.\mvnw clean package -DskipTests
```

Purpose:

- Cleans old build artifacts.
- Compiles application.
- Creates executable JAR.
- Skips unit tests for faster packaging.

Build Docker Image

```
docker build -t devanurag/student-info:v5 .
```

Purpose:

- Creates Docker image from Dockerfile.
- Tags image as version v5.

Verify Image

```
docker images
```

Displays all locally available Docker images.

Push Image to Docker Hub

```
docker push devanurag/student-info:v5
```

Uploads image to Docker Hub for Kubernetes deployment.

---

3. Running Application Locally Using Docker

Create Docker Network

```
docker network create student-network
```

Purpose:

Allows containers to communicate using container names.

Run MySQL Container (PowerShell)

```
docker run -d ^
--name mysql ^
--network student-network ^
-e MYSQL_ROOT_PASSWORD=student ^
-e MYSQL_DATABASE=studentdb ^
-p 3306:3306 ^
mysql:8.0
```

Run MySQL Container (WSL)

```
docker run -d
  --name mysql
  --network student-network
```

```
  -e MYSQL_ROOT_PASSWORD=root
  -e MYSQL_DATABASE=studentdb
  -e MYSQL_USER=studentuser
  -e MYSQL_PASSWORD=studentpass
  -p 3306:3306
  mysql:8.0
```

Verify MySQL Startup

```
docker logs -f mysql
```

Purpose:

Follow startup logs and confirm MySQL is ready.

Run Spring Boot Container

```
docker run -d
  --name springboot
  --network student-network
  -p 8080:8080
  student-app:v1
```

Verify Running Containers

```
docker ps -a
```

Remove Containers

```
docker rm -f springboot mysql
```

Access Application

Because WSL forwards ports to Windows automatically:

```
http://localhost:8080/students
```

---

4. Kubernetes Deployment Verification

Check Pods

```
kubectl get po -n student-namespace
```

or

```
kubectl get pods -n student-namespace
```

Watch Pod Status Continuously

```
kubectl get pods -n student-namespace -w
```

Useful during deployment and autoscaling.

Check Deployments

```
kubectl get deploy -n student-namespace
```

Describe Deployment

```
kubectl describe deploy student-api -n student-namespace
```

Shows:

- Replica count
- Resource requests
- Events
- Rollout history

Check Rollout Status

```
kubectl rollout status deployment/student-api -n student-namespace
```

Confirms successful deployment rollout.

---

5. Pod Failure & Self-Healing Demonstration

Delete API Pod:

```
kubectl delete pod <api-pod> -n student-namespace
```

Delete MySQL Pod:

```
kubectl delete pod <mysql-pod> -n student-namespace
```

Observation:

- Deployment immediately creates replacement pods.
- Demonstrates Kubernetes Self-Healing.

---

6. Understanding Services

Check Services

```
kubectl get svc -n student-namespace
```

Service Selector

Example:

```
selector:
  app: mysql
```

Matches pods with:

```
labels:
  app: mysql
```

Flow:

Service ↓ Find Pods with app=mysql ↓ Route Traffic

Benefits:

- Stable Endpoint
- Load Balancing
- Service Discovery

---

7. Kubernetes DNS

Inside the cluster, applications communicate using service names.

Instead of:

```
10.48.1.23
```

Use:

```
mysql-service
```

Flow:

mysql-service ↓ Kubernetes DNS ↓ Cluster IP ↓ MySQL Pod

Advantages:

- No hardcoded IP addresses
- Automatic service discovery

---

8. Persistent Storage

Problem Without Persistence

Suppose MySQL stores:

1 John 2 Alice 3 Bob

Delete Pod:

```
kubectl delete pod mysql-pod
```

New Pod starts.

Result:

All data lost.

Reason:

Pod storage is ephemeral.

Persistent Volume Concepts

Physical Storage:

Google Persistent Disk

Kubernetes Abstraction:

Persistent Volume (PV)

Application Request:

Persistent Volume Claim (PVC)

Dynamic Provisioning in GKE

Traditional Flow:

PVC → PV → Disk

GKE Flow:

PVC ↓ StorageClass ↓ PV ↓ Google Persistent Disk

No manual PV creation required.

---

9. MySQL Data Storage

MySQL stores data inside:

```
/var/lib/mysql
```

Contains:

- Database files
- Tables
- Indexes
- Logs

Volume Mount

```
volumeMounts:
- name: mysql-storage
  mountPath: /var/lib/mysql
```

Meaning:

Make storage available inside container.

Volume

```
volumes:
- name: mysql-storage
  persistentVolumeClaim:
    claimName: mysql-pvc
```

Meaning:

Use storage from PVC.

Complete Flow

mysql-pvc ↓ volume ↓ volumeMount ↓ /var/lib/mysql

Result:

Database survives pod restarts.

---

10. Database Initialization Strategy

Init SQL Script

Database schema and seed data are stored in:

```
init.sql
```

Kubernetes Storage Method

Store SQL file inside ConfigMap.

Flow:

ConfigMap ↓ Mounted File ↓ /docker-entrypoint-initdb.d/init.sql

---

MySQL Startup Behavior

MySQL Container Starts ↓ Database Empty? ↓ YES ↓ Execute SQL Scripts ↓ Create Tables ↓ Insert Records

Important:

Spring Boot does not insert initial data.

This avoids duplicate data when multiple replicas run.

---

11. Secrets Management

List Secrets:

```
kubectl get secrets -n student-namespace
```

Used for:

- Database Passwords
- Usernames
- Sensitive Configuration

---

12. Node Monitoring

Check Node Resource Usage

```
kubectl top nodes
```

Describe Node

```
kubectl describe node <node-name>
```

Example:

```
kubectl describe node gke-student-info-cluster-default-pool-39102899-e9qy
```

Provides:

- CPU allocation
- Memory allocation
- Running Pods
- Events

---

13. Horizontal Pod Autoscaler (HPA)

Purpose:

Automatically scale pods based on CPU utilization.

Verify Metrics

```
kubectl top pods -n student-namespace
```

```
kubectl top nodes
```

```
kubectl get hpa -n student-namespace
```

Watch HPA

```
kubectl get hpa -n student-namespace -w
```

Observe:

- Current CPU
- Desired Replicas
- Current Replicas

---

14. Generate Load for HPA

PowerShell:

```
1..10000 | % { Invoke-WebRequest http://8.233.153.189/students }
```

or

```
1..10000 | % { Invoke-WebRequest http://8.233.153.189/students -
UseBasicParsing | Out-Null }
```

or

```
1..10000 | % { irm http://8.233.153.189/students | Out-Null }
```

HPA Demonstration Steps

Step 1

```
kubectl get hpa -n student-namespace -w
```

Step 2

Generate load.

Step 3

```
kubectl top nodes
```

Step 4

Observe replicas increasing automatically.

---

15. Ingress

Verify Ingress

```
kubectl get ingress -n student-namespace
```

Why Service Before Ingress?

Ingress cannot directly communicate with pods.

Incorrect:

Ingress ✖ Pods

Correct:

Ingress ↓ Service ↓ Pods

Internal Role of Service

Service responsibilities:

- Pod Discovery
- Load Balancing
- Stable DNS

Path Based Routing

Example:

Internet ↓ Ingress

/students → student-api-service

/orders → order-service

/claims → claims-service

Single Public IP.

Multiple Applications.

Why Companies Prefer Ingress

Without Ingress:

4 Services ↓ 4 Load Balancers ↓ 4 Public IPs

Higher Cost

With Ingress:

1 Load Balancer ↓ 1 Public IP ↓ Multiple Applications

Much Cheaper

GKE Ingress Behind the Scenes

Creating an Ingress automatically creates:

- Google Load Balancer
- Public IP
- Health Checks
- Forwarding Rules

No manual configuration required.

Why Use Rules Instead of defaultBackend?

Answer:

Rules support path-based routing and make the architecture extensible for future microservices.

Even if only one service exists today, future services can be added without redesigning the Ingress.

---

16. Resource Requests & Limits

Initial Configuration:

```
requests:
  cpu: 200m
```

```
  memory: 256Mi

limits:
  cpu: 500m
  memory: 512Mi
```

Issue:

Pods failed scheduling due to insufficient CPU.

Optimized Configuration:

```
requests:
  cpu: 100m
  memory: 256Mi

limits:
  cpu: 500m
  memory: 512Mi
```

Result:

- Better scheduling
- Stable performance
- Efficient cluster utilization

---

17. Key Kubernetes Features Demonstrated

1. Containerization using Docker
2. Kubernetes Deployments
3. Replica Management
4. Self-Healing Pods
5. Service Discovery
6. Kubernetes DNS
7. Secrets Management
8. Persistent Storage using PVC
9. Dynamic Provisioning in GKE
10. ConfigMap-based Database Initialization
11. Horizontal Pod Autoscaling
12. Ingress Routing
13. Load Balancing
14. Resource Requests and Limits
15. High Availability
16. Scalability
17. Fault Tolerance

Final Architecture Summary

Internet ↓ GCP Load Balancer ↓ Ingress ↓ student-api-service (ClusterIP) ↓ 4 Spring Boot Pods ↓ mysql-service (ClusterIP) ↓ MySQL Pod ↓ PVC ↓ Google Persistent Disk

This architecture provides:

- High Availability
- Scalability
- Self-Healing
- Persistent Storage
- Secure Configuration
- Cost-Effective External Access
- Cloud-Native Deployment